

Introduction

- A Greedy Algorithm is a problem-solving method that builds a solution step by step, always choosing the best option available at the current step, hoping it leads to the global optimum (the best overall solution).
- It never looks back or revises its decisions.
- It's like making a series of locally optimal choices.

Analogy

- You have a bag of candies of different sizes.
- You always eat the biggest candy first, because it gives the most satisfaction right now.
- You never think if saving it for later might give you a better outcome.
- That's how greedy algorithms think, "take the best now."
- It doesn't think about the overall picture or what might happen later; it just keeps choosing the most "profitable" or "beneficial" local decision in the hope that, by the end, all these small best choices add up to the globally best solution.

How the Greedy Approach Works

Problem - You are given coin denominations, say [1, 5, 10, 25], and you need to make change for a certain amount, for example, 63 cents, using the fewest number of coins.

To design a greedy algorithm, you go through five mental steps:

1. Understand the problem - What are we trying to minimize or maximize? (Time, cost, number of items, etc.) Here, we are trying to minimize the number of coins used to make a total of 63 cents. We can use coins of 1, 5, 10, and 25 cents.
2. Define the greedy rule - What does "best choice right now" mean for this problem? Always pick the largest coin that is less than or equal to the remaining amount.
3. Sort or organize the data so you can make your greedy choices easily, in this scenario [25, 10, 5, 1].
4. Pick the next best item that doesn't break any rule or constraint. Total coins used: 6 (two 25s, one 10, three 1s)

5. Repeat until you reach your target or there are no more valid moves. We stop when the remaining amount = 0. The greedy process worked perfectly here, we found a valid, minimal solution.

Greedy is not Always Equal Optimal

- The greedy method doesn't always guarantee the best solution for every problem.
- For example, if your coin denominations were [1, 3, 4] and you wanted to make 6, the greedy algorithm would pick:
 - 4 (remaining 2)
 - 1 (remaining 1)
 - 1 (remaining 0) - 3 coins
- But the optimal solution is two 3s (only 2 coins).
- So greedy can fail when local choices don't lead to a global optimum.

Example 1: Coin Change Problem (Greedy)

- Let's say you have coins of values 25, 10, 5, and 1, and you want to make a total of 63.
- The greedy rule here is, "Always pick the largest coin that does not exceed the remaining amount"
- Let's walk through it:
 - Start with 63. The largest coin ≤ 63 is 25, so take it.
Remaining = 38.
 - Again, 25 ≤ 38 - take another 25.
Remaining = 13.
 - Next, 10 ≤ 13 - take 10.
Remaining = 3.
 - Then 1 ≤ 3 - take 1 three times.
- Your coins are: 25, 25, 10, 1, 1, 1. That's a total of 6 coins. $63 - 25 - 25 - 10 - 1 - 1 - 1 = 63$

- This works perfectly because our coin system is “canonical” - it’s designed so that greedy works.
- If the coin values were irregular, greedy might fail.

```
def greedy_coin_change(coins, amount):
    coins.sort(reverse=True)
    result = []
    for coin in coins:
        while amount >= coin:
            amount -= coin
            result.append(coin)
    return result

coins = [25, 10, 5, 1]
amount = 63
print("Coins used:", greedy_coin_change(coins, amount))
```

Example 2: Activity Selection Problem

- You have several activities in a day, each with a start and end time.
- You want to attend as many as possible without overlapping.
- Let’s take these activities:
 A1: 1 - 3
 A2: 2 - 5
 A3: 4 - 6
 A4: 6 - 7
 A5: 5 - 8
 A6: 8 - 9
- The greedy rule here is, “Always choose the activity that finishes earliest and skip overlapping ones”

So:

- Pick A1 (ends at 3)
- Next, pick A3 (starts after 3)
- Then pick A4 (starts after 6)
- Finally pick A6 (starts after 8)

You can attend 4 activities: A1, A3, A4, A6.

Time: 0 - 1 - 2 - 3 - 4 - 5 - 6 - 7 - 8 - 9

A1: [1---3]

A2: [2-----5]

A3: [4---6]

A4: [6-7]

A5: [5-----8]

A6: [8-9]

```
def activity_selection(activities):
    activities.sort(key=lambda x: x[1]) # sort by end time
    selected = []
    last_end = 0

    for start, end in activities:
        if start >= last_end:
            selected.append((start, end))
            last_end = end

    return selected

activities = [(1,3),(2,5),(4,6),(6,7),(5,8),(8,9)]
print("Selected activities:", activity_selection(activities))
```

Example 3: Fractional Knapsack Problem

You are a thief with a bag that can carry 50kg.

You have items with weights and values:

Gold - value 60, weight 10

Silver - value 100, weight 20

Diamond - value 120, weight 30

You can take fractions of items.

The greedy rule:

Pick items with the highest value-to-weight ratio first.

Let's calculate ratios:

- Gold: $60 / 10 = 6$
- Silver: $100 / 20 = 5$
- Diamond: $120 / 30 = 4$

Pick in order: Gold - Silver - Diamond.

You fill your bag as follows:

- Take Gold (10kg) - value +60 - remaining capacity 40kg.
- Take Silver (20kg) - value +100 - remaining capacity 20kg.
- Take 20kg out of 30kg of Diamond - gain value $(120 * 20/30) = 80$.

Total value = $60 + 100 + 80 = 240$.

Knapsack (50kg): | Gold(10) | Silver(20) | 2/3 Diamond(20) |

Work on the Daily Challenge - Market Vendor(Fractional Knapsack)

Applications of Greedy Algorithms

Greedy algorithms are used when problems can be solved by making a sequence of choices, each one being locally optimal, with the hope that these local choices lead to a globally optimal

solution. They are especially useful in optimization problems, when we want to minimize cost, time, or distance, or maximize value, profit, or efficiency.

1. **Coin Change Problem** - In the coin change problem, the goal is to make a specific amount using the fewest coins possible. The greedy algorithm picks the largest denomination that does not exceed the remaining amount. Though it doesn't always work for all currency systems, it works perfectly in systems like the U.S. or Kenyan coin denominations, making it practical for vending machines, ATMs, and payment systems.
2. **Activity Selection and Scheduling** - In scheduling tasks or events, the greedy approach helps choose the maximum number of non-overlapping activities. The algorithm sorts all tasks by their finishing times and selects each task that starts after the previous one finishes. This principle is used in CPU scheduling, meeting room booking systems, and time management applications.
3. **Fractional Knapsack Problem** - The fractional knapsack problem demonstrates how greedy strategies can achieve optimal results when partial selections are allowed. It selects items with the highest value-to-weight ratio, taking whole or fractional amounts until the capacity is full. This concept is applied in resource allocation, financial investments, and cargo optimization, where maximizing return within constraints is crucial.
4. **Minimum Spanning Tree (MST)** - A minimum spanning tree connects all nodes in a weighted graph with the smallest possible total edge weight. Greedy algorithms such as Prim's and Kruskal's are used here, they repeatedly pick the smallest edge that doesn't form a cycle until all nodes are connected. This approach is widely used in network design, for example, in laying cables, pipelines, or communication networks at minimum cost.
5. **Shortest Path Problems** - Greedy algorithms are also used to find the shortest path between nodes in a graph. Dijkstra's Algorithm is the most common example, it selects the nearest unvisited node at each step, expanding outward until it finds the shortest distance to all others. This concept powers GPS navigation, network routing, and map systems like Google Maps, helping find efficient routes.

Advantages of Greedy Algorithms

1. Simple and intuitive, easy to design and understand.
2. Fast and efficient, most greedy algorithms run in linear or logarithmic time.
3. Low memory usage, decisions are made on the spot without storing complex states.
4. Ideal for real-time or large-scale problems like routing, scheduling, and optimization.

Limitations of Greedy Algorithms

1. May give incorrect results when the problem doesn't have the greedy choice property (like non-canonical coin systems).
2. No backtracking, once a choice is made, it cannot be undone.
3. No guarantee of global optimality